LOCAL DEVOPS MONITORING & ALERTING SANDBOX

Project Report | AWS EC2 • Prometheus • Grafana • Node Exporter • Alertmanager

1. Introduction

This project implements an all-in-one monitoring and alerting environment on an Ubuntu 24.04 AWS EC2 virtual machine. It brings together open-source tools to collect host-level metrics, query and visualize system performance, and manage alert notifications. The setup is intended as a practical DevOps learning sandbox for observing a cloud-hosted Linux server.

2. Abstract

The monitoring stack uses Node Exporter to expose Linux system metrics, Prometheus to scrape and evaluate those metrics, Grafana to display dashboards, and Alertmanager to receive and manage alerts. Prometheus is configured with rules for high CPU usage, high disk usage, and monitoring-target availability. The configuration files, dashboard export, installation script, README, and evidence screenshots are maintained in a GitHub repository.

3. Objectives

- Collect CPU, memory, disk, and host metrics from the EC2 instance.
- Configure Prometheus scraping for Prometheus itself and Node Exporter.
- Visualize server metrics through a Grafana Node Exporter Full dashboard.
- Define alert rules for CPU, disk, and target availability conditions.
- Validate Alertmanager alert reception and resolution using amtool.
- Version-control the configuration and project documentation in GitHub.

4. Tools Used

Tool	Purpose
AWS EC2 / Ubuntu 24.04	Cloud virtual machine and Linux operating system
Prometheus	Metric scraping, querying, and rule evaluation
Node Exporter	Exposes host-level system metrics
Grafana	Metrics visualization and dashboarding
Alertmanager	Receives, groups, and manages alerts
Bash, Git, GitHub	Installation automation, version control, and project hosting

5. Steps Involved in Building the Project

1. Created an Ubuntu 24.04 EC2 instance and prepared the Linux environment with required utilities.
2. Installed Prometheus, Node Exporter, Alertmanager, and Grafana; enabled the services to start automatically.
3. Configured Prometheus to scrape its own metrics and Node Exporter at 15-second intervals.
4. Created and validated alert rules for CPU usage above 80%, root filesystem usage above 80%, and unavailable monitoring targets.
5. Configured Alertmanager and validated its configuration with amtool.
6. Added Prometheus as a Grafana data source and imported the Node Exporter Full dashboard.
7. Verified Prometheus targets were UP, reviewed Grafana CPU/memory/disk panels, and sent a test alert to Alertmanager.
8. Published configuration files, the dashboard JSON, installation script, README, and screenshots in GitHub.

6. Validation and Results

The Prometheus configuration and alert rules passed promtool validation, and the Alertmanager configuration passed amtool validation. Both Prometheus scrape targets (Prometheus and Node Exporter) were verified as UP. The Grafana dashboard displayed live host metrics; at the time of capture it showed approximately 1.1% CPU busy, 22.3% RAM used, and 27.4% root filesystem used. A SandboxTestAlert was successfully submitted to Alertmanager and appeared in the active-alert listing. The monitoring services were verified as running.

7. Conclusion

The project successfully demonstrates a centralized monitoring and alerting stack on AWS EC2 using open-source DevOps tools. It provides hands-on experience with Linux service management, Prometheus configuration and alert rules, Grafana dashboards, Alertmanager testing, and Git-based documentation. The repository can be reused as a starting point for more advanced monitoring, notification integrations, and infrastructure automation.

GitHub Repository: github.com/chinnu9729/devops-monitoring-sandbox